

Ragged and multidimensional arrays

CSC103 Programming Fundamentals / Lecture 23

Dr. Muhammad Farhan

Associate Professor of Computer Science
Department of Computer Science
COMSATS University Islamabad
Sahiwal Campus

Fall 2026

The problem for this lecture

Write a method to count all elements in a non-null ragged array with non-null rows. Test $\{\{1\},\{2,3\},\{\}\}$.

Assume the outer array and every row are non-null.

Learning objectives

- Pass a two-dimensional array to a method
- Handle ragged rows
- Explain copying and additional dimensions

CLO-2

Two-dimensional parameters

Ragged arrays

Shallow and deeper copying

Additional dimensions

Two-dimensional parameters

- A method can accept `int[][]` in its parameter list.
- As with other arrays, the reference value is copied.
- Element changes can affect caller-visible data.

Example

```
static int total(int[] [] data) {  
    int sum = 0;  
    for (int[] row : data)  
        for (int v : row) sum += v;  
    return sum;  
}
```

Ragged arrays

- Rows may have different lengths.
- Allocate the outer array and each row separately when needed.
- A newly created outer reference array starts with null rows.

Example

```
int[] [] groups = new int[2] [];  
groups[0] = new int[] {1};  
groups[1] = new int[] {2, 3, 4};
```

Shallow and deeper copying

- Copying only the outer array preserves shared row references.
- Changing a shared row element affects both structures.
- Independent rows require copying each row too.

Example

```
int[] [] copy = original.clone();  
copy[0][0] = 99;  
// original[0][0] is now 99
```

Additional dimensions

- Each extra dimension introduces another array level.
- Give every index a domain meaning.
- Prefer the smallest number of dimensions that expresses the task.

Example

```
int[] [] [] attendance = new int[2][3][4];  
// [section][week][session]
```

A ragged array needs a row-specific bound

- The outer length counts row references, not every stored element.
- Each non-null row supplies its own length.
- Adding the row lengths counts all primitive elements, including zero for an empty row.

Example

`{{1},{2,3},{}}` has row lengths 1,2,0.

Element count = $1 + 2 + 0 = 3$.

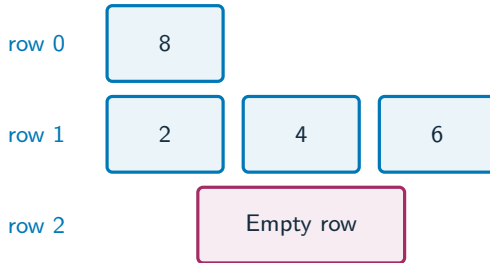
Copy depth follows the structure

- Cloning a two-dimensional array copies the outer row references.
- Cloning each row as well produces separate primitive element storage.
- A deeper structure may require further copying decisions.

Example

```
int[][] copy = new int[a.length][];  
for (int r=0; r<a.length; r++)  
    copy[r] = a[r].clone();
```

Ragged rows have different lengths



What to notice

The outer length counts rows; each row supplies its own element count.

Key distinctions

Row	Contents	Length	Valid indexes
0	{1}	1	0
1	{2,3}	2	0,1
2	{}	0	None

These distinctions determine which operation or control structure fits the problem.

First example: methods

```
static int total(int[] [] data) {  
    int sum = 0;  
    for (int[] row : data)  
        for (int v : row) sum += v;  
    return sum;  
}
```

First example: execution

```
int[] [] data = {{1, 2}, {3}};  
System.out.println(total(data));  
int[] [] copy = data.clone();  
copy[0][0] = 9;  
System.out.println(data[0][0]);
```

First example: result

6

9

The total method processes unequal row lengths.
clone copied the outer references, so the first row remains
shared.

Shallow and deeper copying
Additional dimensions

Guided practice

Write a method to count all elements in a non-null ragged array with non-null rows. Test `{{1},{2,3},{}}`.

Attempt the solution before continuing.
Use the definitions and examples from this lecture.

Solution strategy

- Assume the outer array and every row are non-null.
- Visit each row and add its length to a counter.
- Return the accumulated count.

The next program uses fixed inputs so its result can be reproduced.

Complete solution

```
public class Solution23 {  
    public static void main(String[] args) throws Exception {  
        int[][] values = {{1}, {2, 3}, {}};  
        System.out.println(elementCount(values));  
    }  
    static int elementCount(int[][] values) {  
        int count = 0;  
        for (int[] row : values) {  
            count += row.length;  
        }  
        return count;  
    }  
}
```

Execution trace

Row visited	Length added	Count before	Count after
0	1	0	1
1	2	1	3
2	0	3	3

Follow the changing state in execution order.

Solution result

3

Why this result follows
Return the accumulated count.

A common incorrect approach

```
int[][] a = new int[3][];  
a[0][0] = 5;
```

Example

a[0] is initially `null`. Allocate the row, for example `a[0]=new int[2]`, before assigning an element.

Boundary and failure checks

Check the stated input assumptions.

Write a method that makes an independent copy of a ragged `int[][]` with non-null rows. Demonstrate that mutation of the copy leaves the original unchanged.

Start `count=0` and add `row.length` for each row. The result is 3. The empty row contributes zero.

Check your understanding

Does outer-array clone duplicate each row? How many values are in `new int[2][3][4]`?

Write a prediction and explain the rule that supports it.

Answer and explanation

No. It shares the row references. The rectangular allocation contains 24 int values.

`a[0]` is initially null. Allocate the row, for example `a[0]=new int[2]`, before assigning an element.

Compare cases and predict outcomes

Row	Values	Length
0	8	1
1	2, 4, 6	3
2	No elements	0

Discuss

Explain each expected result before running the example. Which case would reveal a wrong assumption?

Apply the idea

Independent practice

For $\{\{8\}, \{2,4,6\}, \{\}\}$, compute the total number of elements and their sum. Assume all rows are non-null.

1. Work through the input by hand.
2. Write or correct the program.
3. Justify the expected result.

Solution and reasoning

```
int[][] data = {{8}, {2, 4, 6}, {}};  
int count = 0, sum = 0;  
for (int[] row : data) {  
    count += row.length;  
    for (int value : row) sum += value;  
}  
System.out.println(count + " " + sum);
```

- The lengths add to 4.
- The values add to 20.
- The empty row contributes zero to both totals without needing a special loop branch.

Selecting the row with the largest sum

- Compute one row sum at a time using that row's length.
- Initialise the best sum from an actual row rather than zero if negatives are allowed.
- A strict $>$ update keeps the first row when sums tie.

Source: [supplied ISB campus slides](#). See [speaker notes and ISB_Source_Map.md](#).

Worked problem: Selecting the row with the largest sum

Task

Find the row with the largest sum in the source matrix $\{\{10,20,30\},\{40,50,60\},\{70,80,90\}\}$.

Input: Values are fixed in the program for a reproducible demonstration.

Before running

Predict the output and identify the variables that change.

Complete ISB example (1/1)

```
public class IsbLecture23 {  
    public static void main(String[] args) throws Exception {  
        int[][] data = {{10,20,30}, {40,50,60}, {70,80,90}};  
        int bestRow = 0;  
        long bestSum = Long.MIN_VALUE;  
        for (int row = 0; row < data.length; row++) {  
            long sum = 0;  
            for (int value : data[row]) sum += value;  
            if (sum > bestSum) { bestSum = sum; bestRow = row; }  
        }  
        System.out.println("Row " + bestRow + ", sum " + bestSum);  
    }  
}
```

Worked trace and expected result

Row index	Row sum	Best row afterward
0	60	0
1	150	1
2	240	2

Expected console output

Row 2, sum 240

Extend the ISB example

Practice

How would you extend the contract to an empty outer array and null rows?

Explain your reasoning

Reject an empty outer array because there is no winning row. Reject null rows or define a separate policy. The demonstrated input is nonempty with non-null rows.

Lecture summary

- a two-dimensional array to a method
 - ragged rows
 - copying and additional dimensions
- Assume the outer array and every row are non-null.
 - Visit each row and add its length to a counter.
 - Return the accumulated count.

After-class practice

Write a method that makes an independent copy of a ragged `int[][]` with non-null rows. Demonstrate that mutation of the copy leaves the original unchanged.

Syllabus reading

Deitel Ch 7

Next lecture: Exception handling